> *"The solutions all are simple — after you have arrived at them. But they're simple only when you know already what they are."* — *Robert M. Pirsig,* Zen and the Art of Motorcycle Maintenance

Spinning Flight and Passive Lift: Spinning flight may appear counterintuitive, yet it offers a route to fault-tolerant and more efficient flight. Published controllers keep a quadrotor flying after rotor loss while it spins at more than 20 rad/s [1]. In my current work, I hypothesize that constantly spinning a drone with wings lowers the spin rate a vehicle needs after a rotor loss, while providing a new lever for optimization and efficient flight, by offloading thrust from small, inefficient propellers to the larger "propeller" that the whole drone may now be modeled as. The validation of fault tolerance aided by the design will be conducted on hardware in November 2026. My master's thesis at **IIIT Hyderabad** is about making spinning flight work across the full stack: airframe, wings, estimator and controller. I designed and prototyped the airframe, and my ICUAS 2026 paper on its wings was a **best paper finalist**; on hardware the wings carried **22%** of the vehicle's weight at 8–10 rad/s.

Finding What Actually Limits a System: Through many design iterations in my solo work, I became familiar with the problems of hardware design and rapid prototyping. Once the build was ready, I hit a stability limit. No controller I implemented, in simulation or on hardware, got past 10 rad/s; everything above that spin rate destabilized. After a month of trying various control methods and a custom controller, hoping to leverage the gyroscopic stability I expected, I saw no progress. I decided that the simulator was the problem and switched from Gazebo to Pegasus with Isaac Sim. In Isaac Sim, I was pleasantly surprised that, when run in pure Python, things began working again. And so I managed to tune both my custom controller and PID to a reasonable level of performance. However, when I ran the same controller with PX4 in the loop, the same plant failed. This time, Pegasus afforded me the ability to compare the ground truth with the estimator's output, and I was able to see that the estimator was diverging. This was a limitation in PX4's estimator that I had not considered; it stemmed from the lag between the IMU and the estimator, as well as a small-angle approximation in the estimator's predictor. After this, I implemented 3 levels of simulation, Level 1 being pure Python, Level 2 being PX4 in the loop with ground truth, and Level 3 being PX4 in the loop with no ground truth, so that I could methodically debug any future failures. The PX4 stack, which I had taken for granted, was limiting me. It made me feel I was working at the edge of what was currently possible, and that bringing this problem to light gave anyone an easy way to fly spinning vehicles with PX4.

The lag: A transport delay τ on a body spinning at ω is a pure rotation of the command by $\omega\tau$, and a rotor lag adds that rotation plus an amplitude loss, so what limits a spinning vehicle is phase, not droop. My first attempt at a proof, a Lyapunov and input-to-state stability analysis of the position, attitude and rate cascade, gave a bound that scaled with the mismatch between commanded and realized torque, where lag enters, and so I added a few terms: an **adaptive compensator** that rotates the command back by an angle estimated from a model-free signal whose derivative with respect to that angle is exactly one, so the plant, inertia and control law cancel, together with an **inversion of the rotor dynamics**. With a realistic 30 ms motor lag, neither part is enough alone, but together they hold hover at 29.5 rad/s in PX4-in-the-loop simulation; hardware validation of that controller is under way as of October 2026. I also kept the whole controller on the **PX4** flight stack instead of a companion computer. I was advised against it; it was harder, but cheaper, lighter, and easier for anyone already flying PX4 to reuse. With it, the vehicle has flown stably at 14.6–15.5 rad/s.

By December 2026, I will wrap up my thesis with a flight analysis with and without the wings, showing fault tolerance, and refine my control theory and implementation to better leverage the vehicle's gyroscopic stability.

Research Interests: Co-Designing Limbs That Stiffen Under and For Load:

Question. Can a limb's geometry and its controller be searched jointly, in simulation and transferred to hardware, so that every candidate design comes with a stability guarantee? I want to answer this for a universal limb that reaches softly and then bears load, for manipulation and for locomotion on rugged terrain, in the spirit of an octopus arm or a primate limb. I am deliberately not fixing the limb's form (continuum, articulated or hybrid); it is part of what the search should decide.

Why stiffness is the design variable. Soft and continuum arms are rarely asked to carry weight, and payload is usually treated as a compromise between compliance and strength [2, 3]. Stiffness can rise *under* load, passively, because of how a segment is built, or be raised *for* load, actively, before the load arrives. On my drone, passive wings carried 22% of the weight, and that changed what the rest of the system had to do. Here, geometry would supply a stiffening floor and the controller would add or remove stiffness on top. For the passive part I see origami patterns [4], tendon routing [3], joints that stiffen under tension, and musculoskeletal structures [5, 6]. Guan et al. show one printed material spanning tissue-like to bone-like stiffness through lattice geometry alone, the kind of geometry-only stiffness I want to build on.

Why the search needs a stability analysis. Della Santina et al. prove stability for a planar soft arm

without cancelling its softness [7]. Their Lyapunov proof relies on a constant, positive-definite stiffness K and a spring force that depends on configuration alone. A geometry-stiffened segment breaks the first assumption, since its deflection depends on the loading mode [3]. If its stiffness still derives from a configuration-only elastic energy, the same energy argument may extend; if it depends on the external load, it does not, and I have not found an analysis of this case. Tendon actuation can also soften a backbone toward buckling [2], making stiffness an input-dependent quantity, an active knob on top of the passive floor. I would use this analysis as a constraint on the search, so that learning explores only designs whose controllers can be certified.

Co-design through sim-to-real search. Geometry sets the stiffness curve the controller must work with, and the controller determines how much of that curve is needed, so I would search over both. Soft-robot co-design remains limited by narrow objectives, sim-to-real gaps and computational cost [8]; Candiello et al. [9] show that a low-dimensional embedding of shape, material and actuation makes joint search tractable. I would add the stability constraint to such a search, one that benchmarks like Co-Design Gym [10], spanning manipulation, locomotion and deformable dynamics, now make feasible to run. This extends my co-design of a spinning quadrotor's wing shape, airframe and control law, where aerodynamic forces and spin-rate limits decided which designs were feasible, and my adaptive control work on partly unknown, state-dependent parameters.

What I bring. A control theorist's habit of finding which assumption actually bounds a system, and proving what happens when it fails. On the drone, the Lyapunov/ISS bound scaled with the torque mismatch where lag enters, and I turned that into an adaptive compensator that estimates an unknown, state-dependent quantity online; a stiffness that depends on load is the same kind of problem. I validate in layers, as with my three-level simulation, and would bring that to soft-robot models and hardware. I have co-optimised controller gains with CMA-ES, and I am now co-optimising the leg length and stiffness of a compliant quadruped with an RL gait policy in Isaac Lab. I have not yet worked with strain-based soft-robot models; learning them is my first step, starting by reproducing the planar stability proof [7] and finding where a load-dependent stiffness breaks it. I can also build what I analyse: I designed, by myself, the spinning quadrotor's airframe, 3D-printed mounts and wings in Fusion 360, sized the wings with BEMT, CFD (ANSYS Fluent) and XFLR5, and flew them; I have also designed PCBs and a CanSat's mechanical subsystems. That would let me iterate on a limb's geometry and test rig myself. I released my ICUAS code and taught PX4, ROS 2 and XFLR5 as a teaching assistant, and I build tools for others to reuse.

References

[1] C. Ke, K.-Y. Cai, and Q. Quan, "Uniform passive fault-tolerant control of a quadcopter with one, two, or three rotor failure," *IEEE Trans. Robot.*, vol. 39, no. 6, pp. 4297–4311, 2023.

[2] Q. Peyron and J. Burgner-Kahrs, "Stability analysis of tendon driven continuum robots and application to active softening," *IEEE Trans. Robot.*, vol. 40, pp. 85–100, 2024.

[3] J. A. Childs and C. Rucker, "Leveraging geometry to enable high-strength continuum robots," *Front. Robot. AI*, vol. 8, Art. no. 629871, 2021.

[4] D. Rus and M. T. Tolley, "Design, fabrication and control of origami robots," *Nat. Rev. Mater.*, vol. 3, pp. 101–112, 2018.

[5] T. Buchner, T. Fukushima, . . . , C. Keplinger, and R. K. Katzschmann, "Electrohydraulic musculoskeletal robotic leg for agile, adaptive, yet energy-efficient locomotion," *Nat. Commun.*, vol. 15, Art. no. 7634, 2024.

[6] Q. Guan, B. Dai, H. H. Cheng, and J. Hughes, "Lattice structure musculoskeletal robots: Harnessing programmable geometric topology and anisotropy," *Sci. Adv.*, vol. 11, no. 29, Art. no. eadu9856, 2025.

[7] C. Della Santina, R. K. Katzschmann, A. Bicchi, and D. Rus, "Model-based dynamic feedback control of a planar soft robot: Trajectory tracking and interaction with the environment," *Int. J. Robot. Res.*, vol. 39, no. 4, pp. 490–513, 2020.

[8] M. Stölzle *et al.*, "Soft yet effective robots via holistic co-design," arXiv:2505.03761, 2025.

[9] V. Candiello, M. Mekkattu, M. Y. Michelis, and R. K. Katzschmann, "A unified low-dimensional design embedding for joint optimization of shape, material, and actuation in soft robots," arXiv:2603.06497, 2026.

[10] A. Newatia, Y. Tsvetkov, L. Pleiss, A. Spielberg, and R. Antonova, "Co-Design Gym: A unified benchmark for embodiment-policy co-optimization," arXiv:2610.02366, 2026.